Day 26 — Real-Time Healthcare Project : Use stateful/window processing for repeated abnormal readings

COMMANDS :

```
cd ~/day23-dstreams

cat > src/main/scala/Day26Healthcare.scala << 'EOF'
import org.apache.spark.SparkConf
import org.apache.spark.broadcast.Broadcast
import org.apache.spark.streaming.{Seconds, StreamingContext}
import scala.util.Try

// ---------- Event schema ----------
case class Vital(patientId: String, heartRate: Int, spo2: Int,
                 systolic: Int, diastolic: Int, temperature: Double)

object Vital {
  // Returns None for malformed lines
  def parse(line: String): Option[Vital] = {
    val a = line.trim.split(",").map(_.trim)
    if (a.length != 6) None
    else Try(Vital(a(0).toUpperCase, a(1).toInt, a(2).toInt, a(3).toInt, a(4).toInt,
a(5).toDouble)).toOption
  }
}

// ---------- Abnormal-vital rules ----------
object Rules {
  type Thr = Map[String, (Double, Double)]   // vital name -> (min, max)

  def reasons(v: Vital, t: Thr): Seq[String] = {
    def chk(name: String, value: Double): Option[String] = {
      val (lo, hi) = t(name)
      if (value < lo) Some(f"$name LOW (value%.1f < $lo%.1f)")
      else if (value > hi) Some(f"$name HIGH (value%.1f > $hi%.1f)")
      else None
    }
    Seq(
      chk("heartRate", v.heartRate),
      chk("spo2", v.spo2),
      chk("systolic", v.systolic),
      chk("diastolic", v.diastolic),
      chk("temperature", v.temperature)
```

```
    ).flatten
  }
}

object Day26Healthcare {
  val RepeatThreshold = 3   // abnormal readings in the window that make a patient CRITICAL

  def main(args: Array[String]): Unit = {
    val conf = new SparkConf().setAppName("Day26Healthcare").setMaster("local[2]")
    val ssc  = new StreamingContext(conf, Seconds(5))
    val sc   = ssc.sparkContext
    sc.setLogLevel("WARN")
    ssc.checkpoint("/tmp/day26-checkpoint")   // needed for stateful ops

    // ---------- Broadcast: thresholds shared read-only with all executors ----------
    val thresholds: Broadcast[Rules.Thr] = sc.broadcast(Map(
      "heartRate"   -> (50.0, 120.0),
      "spo2"        -> (92.0, 100.0),
      "systolic"    -> (90.0, 140.0),
      "diastolic"   -> (60.0, 90.0),
      "temperature" -> (36.0, 38.0)
    ))

    // ---------- Accumulators: counters updated by tasks, read on the driver ----------
    val validAcc     = sc.longAccumulator("validReadings")
    val malformedAcc = sc.longAccumulator("malformedReadings")
    val abnormalAcc  = sc.longAccumulator("abnormalReadings")

    // ---------- Process the vital stream ----------
    val lines  = ssc.socketTextStream("localhost", 9999).filter(_.trim.nonEmpty)
    val parsed = lines.map(Vital.parse)

    parsed.foreachRDD { (rdd, time) =>
      println(s"\n##### BATCH $time #####")
      rdd.foreach { o => if (o.isDefined) validAcc.add(1) else malformedAcc.add(1) }
    }

    val vitals = parsed.flatMap(_.toList)

    // ---------- Abnormal-vital alerts ----------
    val alerts = vitals
      .map(v => (v, Rules.reasons(v, thresholds.value)))
      .filter(_._2.nonEmpty)
```

```
    alerts.foreachRDD { rdd =>
      rdd.foreach(_ => abnormalAcc.add(1))
      rdd.collect().foreach { case (v, rs) =>
        println(s"ALERT ${v.patientId}: ${rs.mkString("; ")}")
      }
      println(s"Totals so far -> valid: ${validAcc.value}, malformed: ${malformedAcc.value},
abnormal: ${abnormalAcc.value}")
    }

    // ---------- STATEFUL: running abnormal-reading count per patient ----------
    val abnormalPerBatch = alerts.map { case (v, _) => (v.patientId, 1) }.reduceByKey(_ + _)
    val runningAbnormal  = abnormalPerBatch.updateStateByKey[Int](
      (newVals: Seq[Int], state: Option[Int]) => Some(state.getOrElse(0) + newVals.sum))

    runningAbnormal.foreachRDD { rdd =>
      println("--- STATEFUL: running abnormal readings per patient ---")
      rdd.collect().sortBy(_._1).foreach { case (p, n) => println(f"$p%-6s $n%3d") }
    }

    // ---------- WINDOW: repeated abnormal readings (30s window, every 10s) ----------
    val abnormalInWindow = alerts
      .map { case (v, _) => (v.patientId, 1) }
      .reduceByKeyAndWindow((a: Int, b: Int) => a + b, Seconds(30), Seconds(10))

    abnormalInWindow.filter(_._2 >= RepeatThreshold).foreachRDD { rdd =>
      rdd.collect().foreach { case (p, n) =>
        println(s"!!! CRITICAL: $p had $n abnormal readings in the last 30s !!!")
      }
    }

    ssc.start()
    println("Day 26 started. Port 9999 (PATIENT,HR,SPO2,SYS,DIA,TEMP)...")
    ssc.awaitTermination()
  }
}
EOF

cd ~/day23-dstreams
rm -rf /tmp/day26-checkpoint
sbt "runMain Day26Healthcare"
```

OUTPUTS :

```
[info] Day 26 started. Port 9999 (PATIENT,HR,SPO2,SYS,DIA,TEMP)...
[info] ##### BATCH 1791782725000 ms #####
[info] Totals so far -> valid: 0, malformed: 0, abnormal: 0
[info] --- STATEFUL: running abnormal readings per patient ---
[info] ##### BATCH 1791782730000 ms #####
[info] Totals so far -> valid: 0, malformed: 0, abnormal: 0
[info] --- STATEFUL: running abnormal readings per patient ---
[info] ##### BATCH 1791782735000 ms #####
[info] Totals so far -> valid: 0, malformed: 0, abnormal: 0
[info] --- STATEFUL: running abnormal readings per patient ---
[info] ##### BATCH 1791782740000 ms #####
[info] Totals so far -> valid: 0, malformed: 0, abnormal: 0
[info] --- STATEFUL: running abnormal readings per patient ---
[info] ##### BATCH 1791782745000 ms #####
[info] Totals so far -> valid: 0, malformed: 0, abnormal: 0
[info] --- STATEFUL: running abnormal readings per patient ---
[info] ##### BATCH 1791782750000 ms #####
[info] Totals so far -> valid: 0, malformed: 0, abnormal: 0
[info] --- STATEFUL: running abnormal readings per patient ---
[info] ##### BATCH 1791782755000 ms #####
[info] Totals so far -> valid: 0, malformed: 0, abnormal: 0
[info] --- STATEFUL: running abnormal readings per patient ---
[info] ##### BATCH 1791782760000 ms #####
[info] Totals so far -> valid: 0, malformed: 0, abnormal: 0
[info] --- STATEFUL: running abnormal readings per patient ---
[info] ##### BATCH 1791782765000 ms #####
[info] Totals so far -> valid: 0, malformed: 0, abnormal: 0
[info] --- STATEFUL: running abnormal readings per patient ---
[info] ##### BATCH 1791782770000 ms #####
[info] Totals so far -> valid: 0, malformed: 0, abnormal: 0
[info] --- STATEFUL: running abnormal readings per patient ---
[info] ##### BATCH 1791782775000 ms #####
[info] Totals so far -> valid: 0, malformed: 0, abnormal: 0
[info] --- STATEFUL: running abnormal readings per patient ---
```